
KubeCon
CloudNativeCon
Europe 2025
LONDON BUS

KubeCon | CloudNativeCon
Europe 2025

Scaling GPU Clusters Without Melting Down!

Ryan Hallisey, NVIDIA
Alay Patel, NVIDIA

Scaling Goals

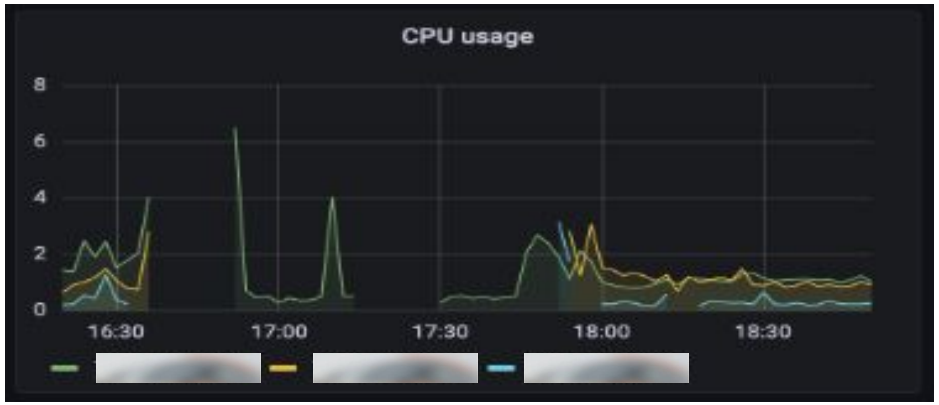
- GPUs are getting more powerful so clouds can support more workloads per chip
- This leads to larger pod count/node count/workload count
- We run baremetal and changing control plane hardware every year is expensive
- We need to scale up keeping the control plane resources constant

Incidents, Incidents, Incidents…

1. large number of list calls to secrets would lead to control plane outages
2. restart of apiserver releases 40% of memory
3. load balancing skew in apiserver creates cascading failures
4. and many more

High secrets count borking clusters!

- If secrets count is high, apiserver would OOM
- After multiple such incidents, we ran some experiments
  - 5k secrets of 0.2MB each
  - 2 or more concurrent list calls ooms the server
- Inefficiency in List call handling: KEP-3157



API Priority & Fairness to the Rescue

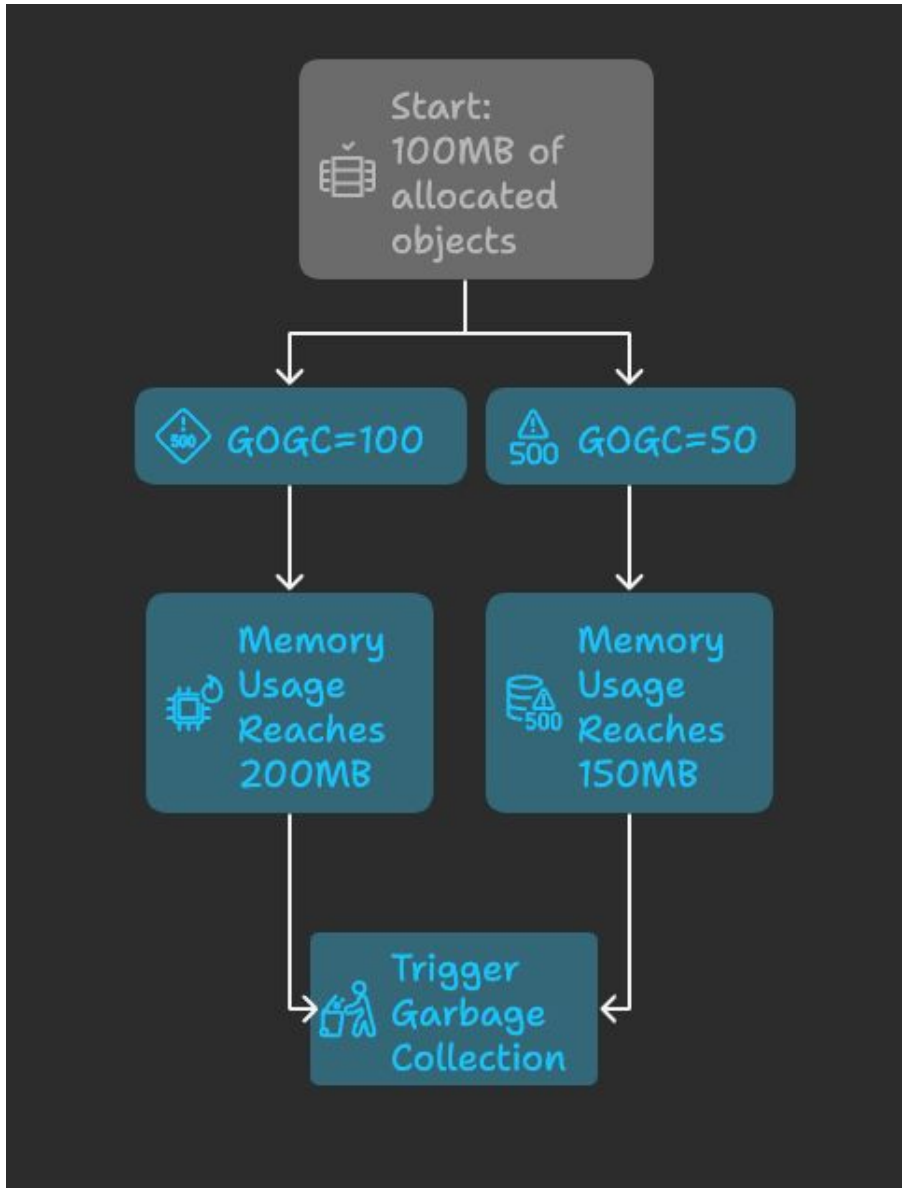
- Use resource quota to limit the number of secrets in cluster
- Use API Priority and Fairness to:
  a. Identify any list request coming to kube-apiserver
  b. only allow two concurrent requests to be executed
     - queue the remaining
- Significantly reduced the OOMs

Taking Care of Your Control Plane With API Priority and Fairness and Resource Quotas - Matteo Ruina & Ayaz Badouraly, Datadog

Observation: restart reduces memory usage

- apiserver restart would free up 40% of memory with the same load
- Started exploring more aggressive GOGC

Garbage collection based on GOGC



Can we tune GOGC?

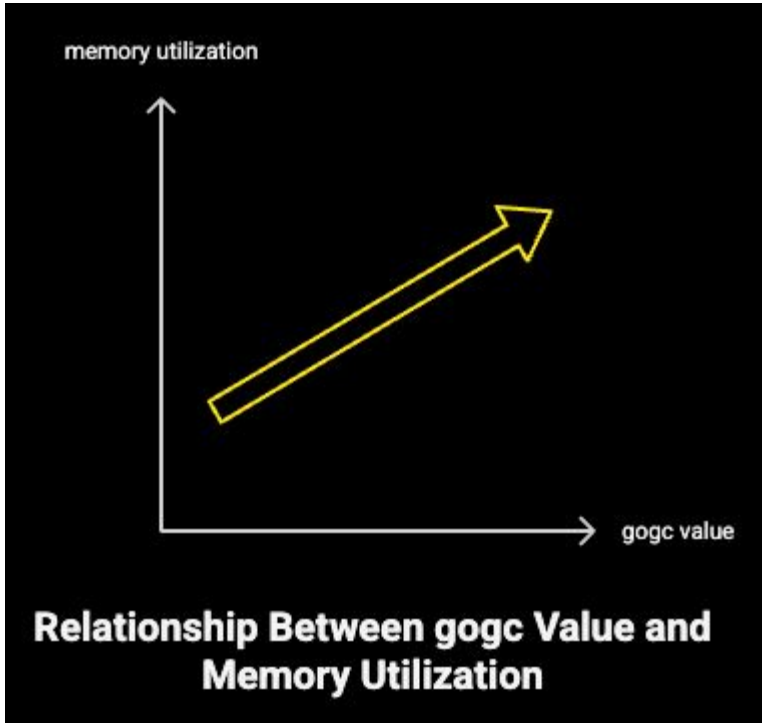
Default:

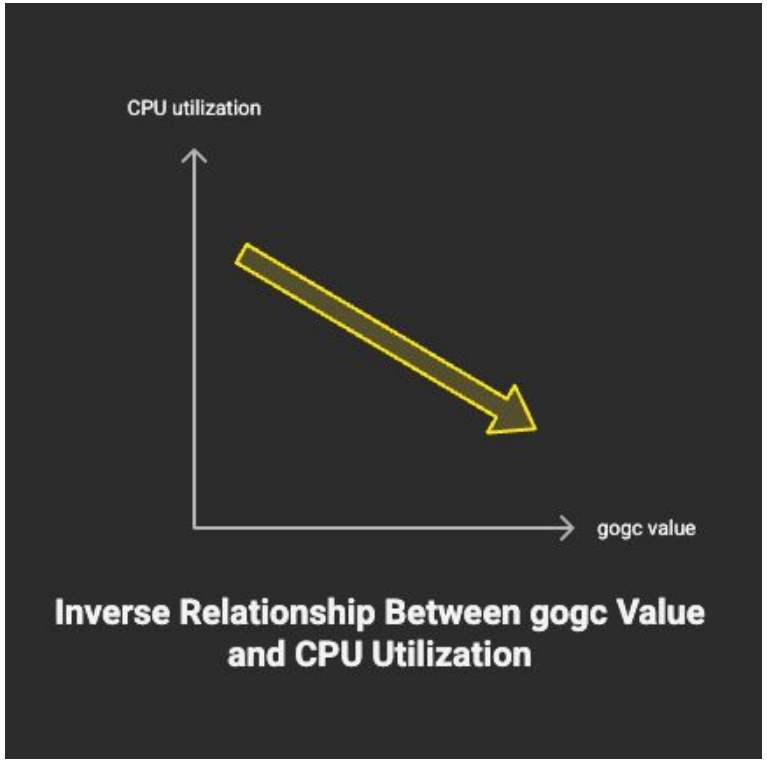
```
env:
- name: GOGC
  value: "100"
```

Tunend:

```
env:
- name: GOGC
  value: "50"
```

GOGC: more resources from the wild!



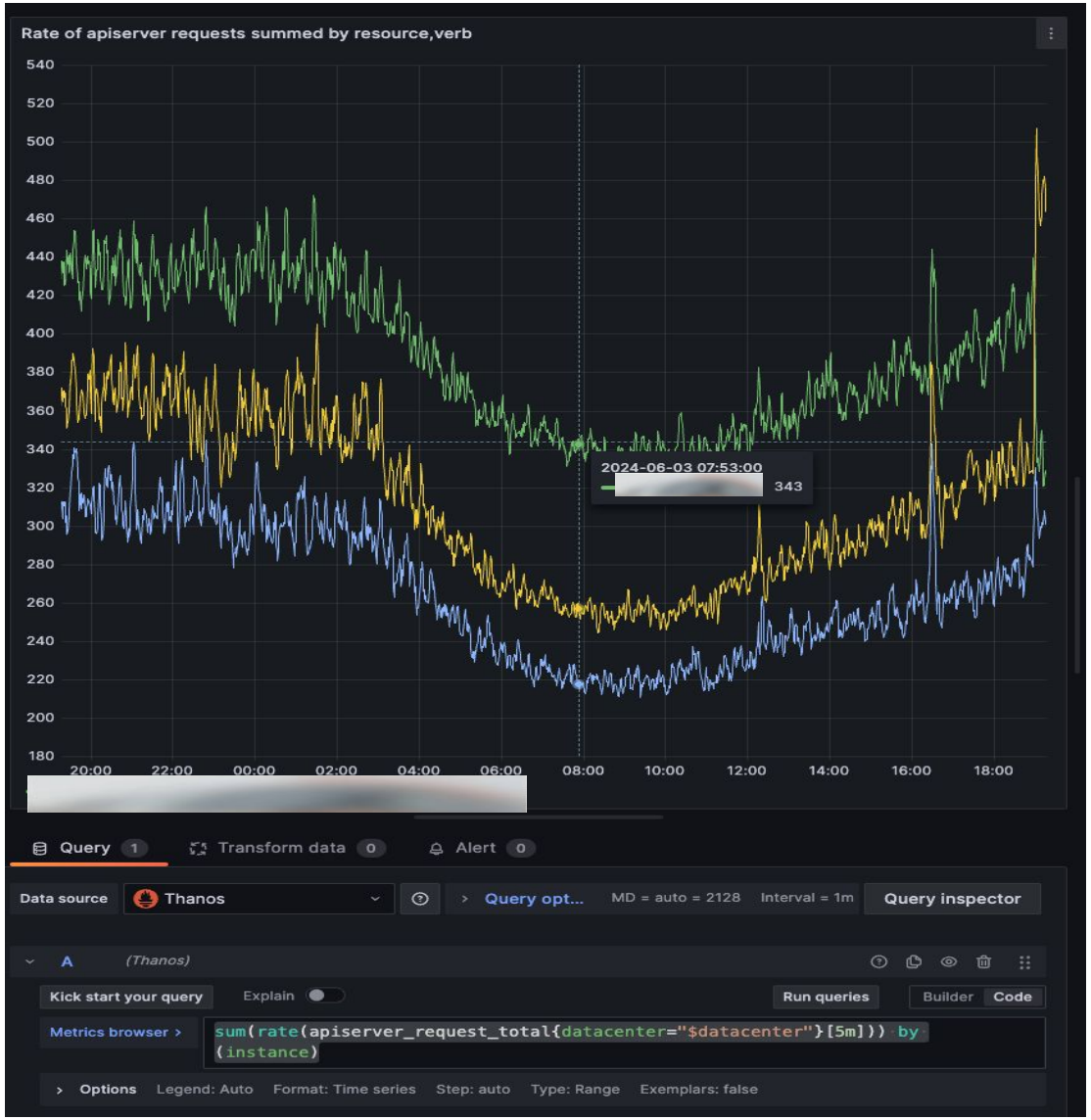


https://www.uber.com/blog/how-we-saved-70k-cores-across-30-mission-critical-services/

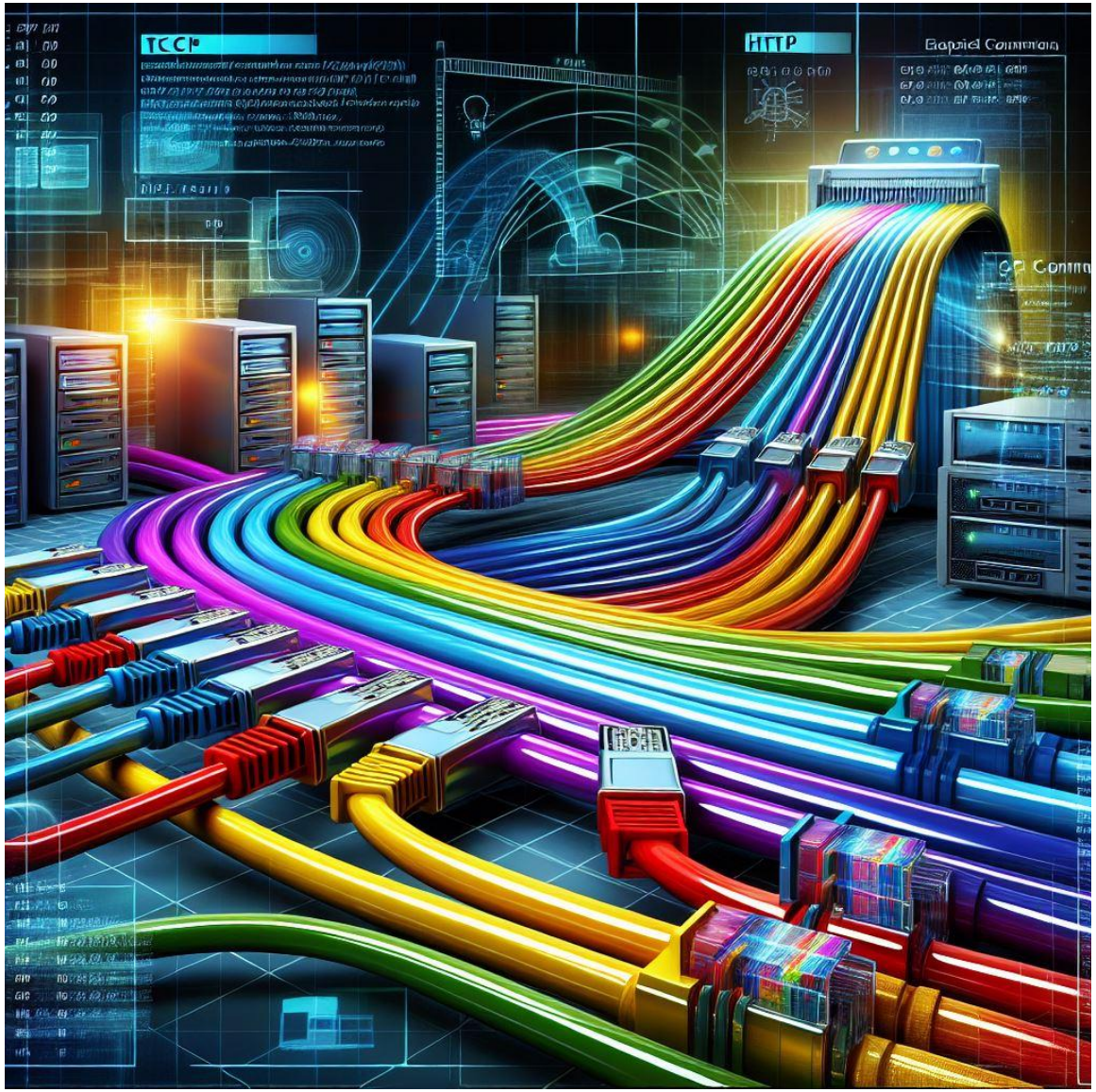
Big skew in apiservers' memory

Skew in TCP connections

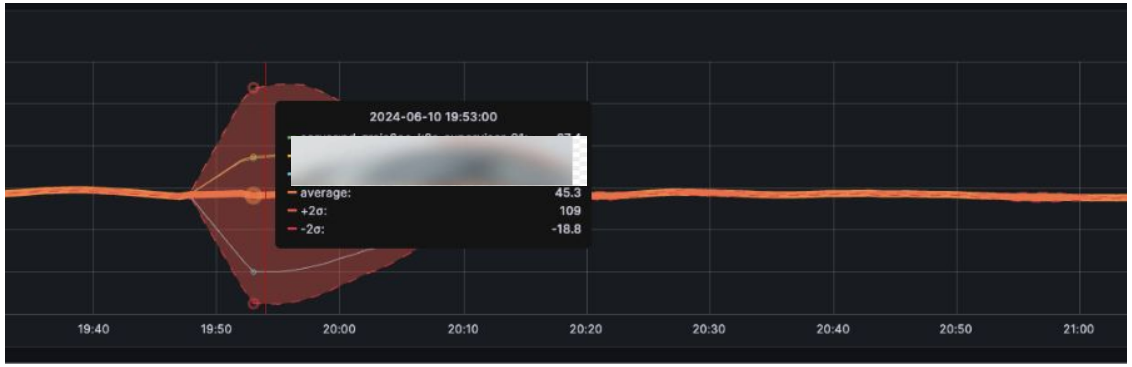
Resultant skew in rate of requests



Solution: configure –goaway-chance



- Requests use http2, sent on established tcp connections
- HAProxy balances load based on connections, not on requests
- Once the connections are established, goaway-chance parameter sends a random connection GOAWAY TCP message
- Allow long standing connections to be balanced
- Avoiding skews in connections lead to much distributions of load.
- https://github.com/kubernetes/kubernetes/pull/88567



Architectural mistakes

- Large size of secret per workload
- Our client that doesn't cache and makes periodic list calls
- Both of the above combined was catastrophic

sig-api-machinery to the rescue

- Watch List
- Consistent cache list
- CBOR binary encoding

Allow informers for getting a stream of data instead of chunking #3157

Open

[Beta: 1.31] Consistent Reads from Cache #2340

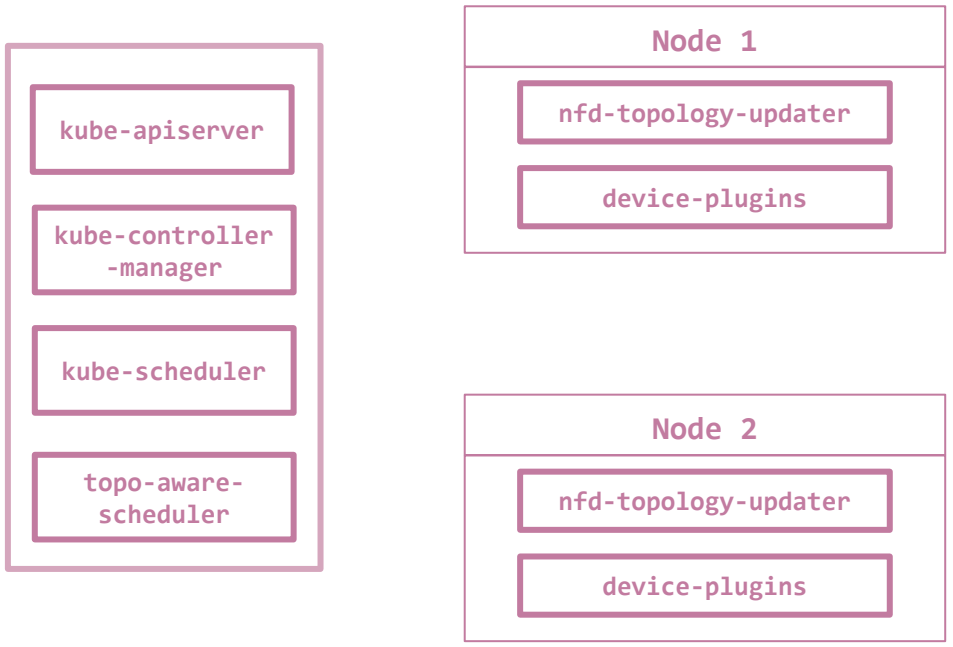
Open

CBOR Serializer #4222

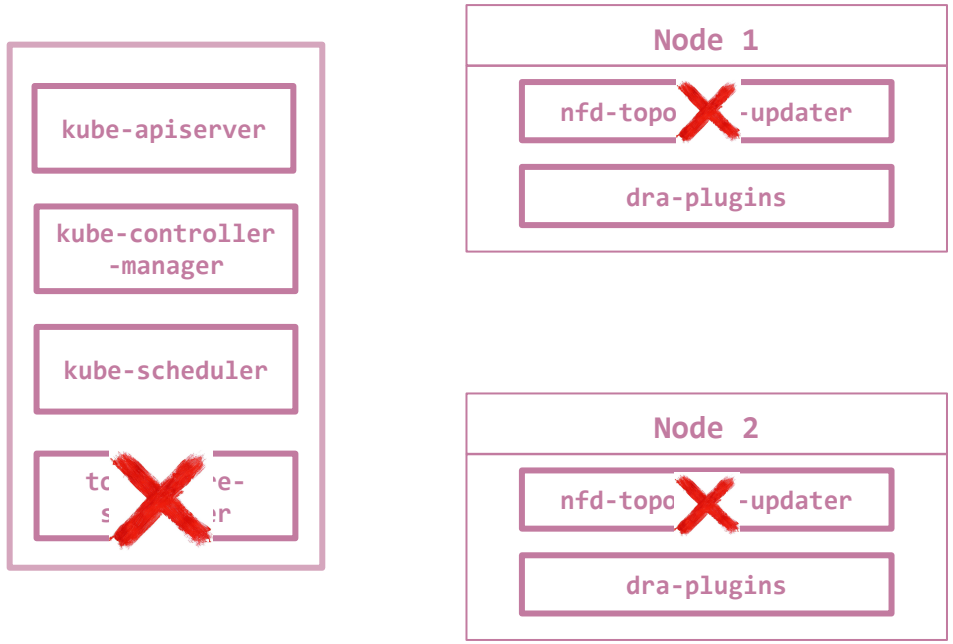
Open

Upcoming…Transition to DRA

NUMA aligned CPU-GPU devices



Future with DRA



kubecon NA 2024

A Tale of 2 Drivers: GPU Configuration on the Fly Using DRA

Simulation testing with KWOK

KWOK (Kubernetes WithOut Kubelet)

- Early performance comparison of both the stacks
- Used KWOK for simulate GPU workloads in cluster
- https://kwok.sigs.k8s.io/

⚡Lightning Talk: Scaling To the Stars: Simulating Massive Clusters With KWOK - Soumya Balakrishnan, NVIDIA

How We Tackle KubeVirt's Growth and Scalability - Ľuboslav Pivarč, Red Hat & Alay Patel, NVIDIA

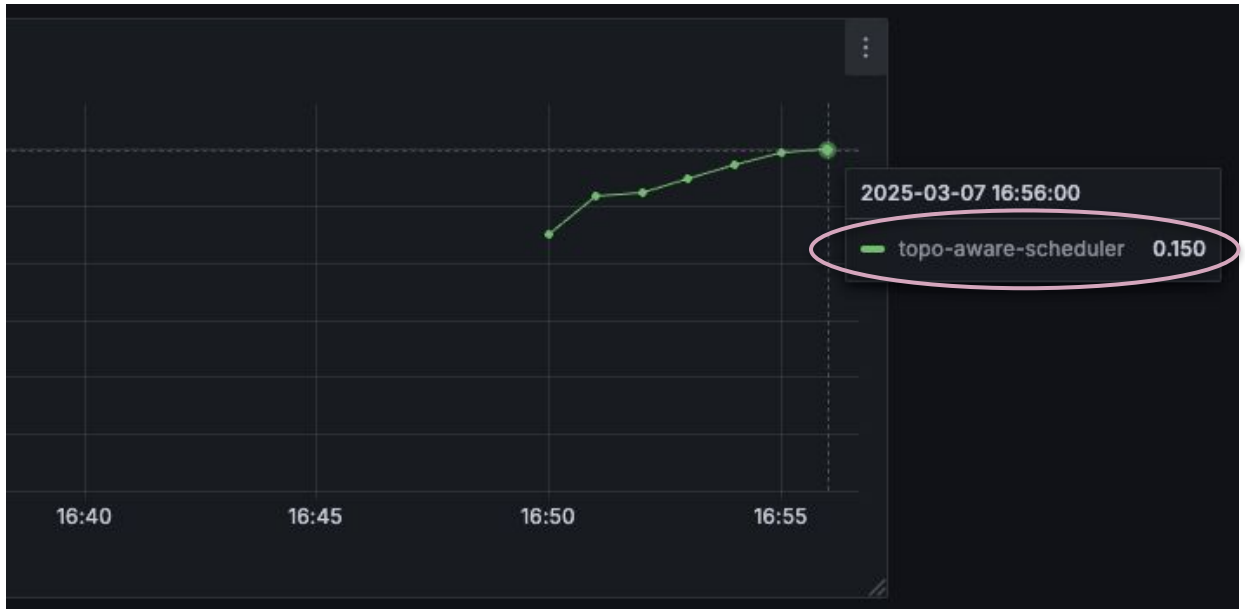
Simulation test Setup

- K8s 1.32 cluster
- kwok for simulation
    - works out of the box
- Scale up 0 -> 4000 with
    i. Topo-aware scheduler
    ii. DRA

Results: Scheduling latency

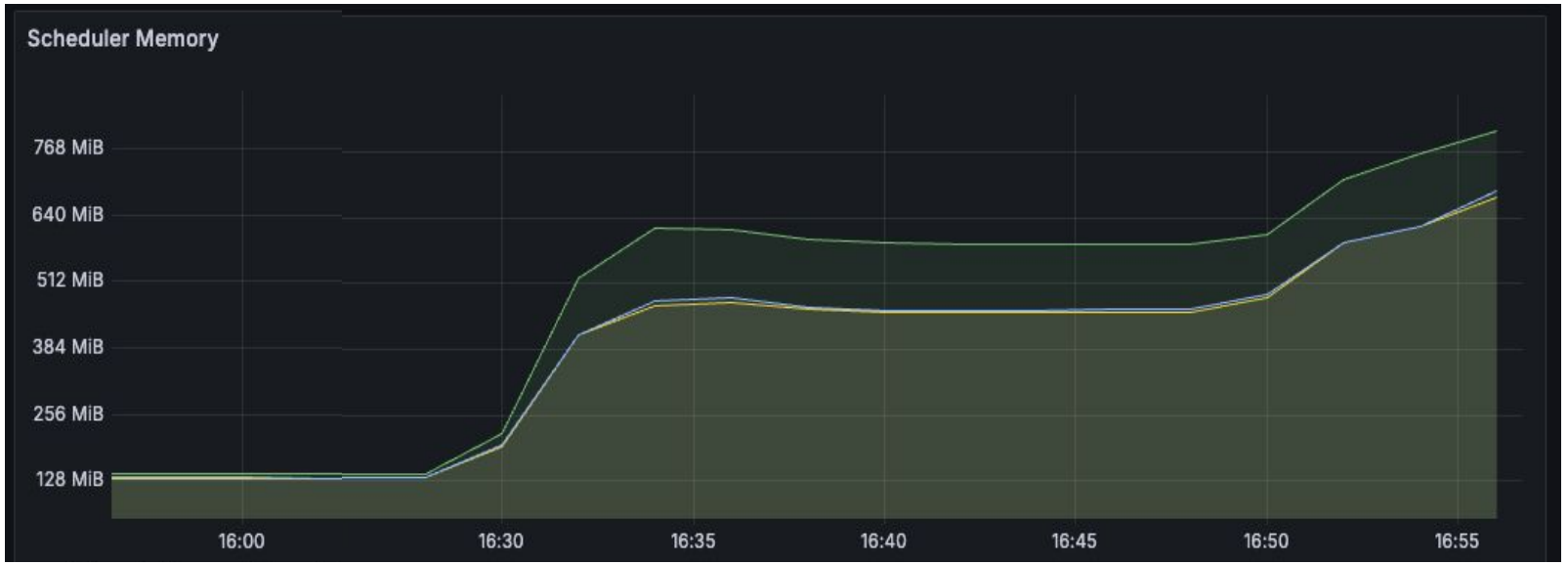
KubeCon CloudNativeCon Europe 2025

topo-aware-scheduler Latency



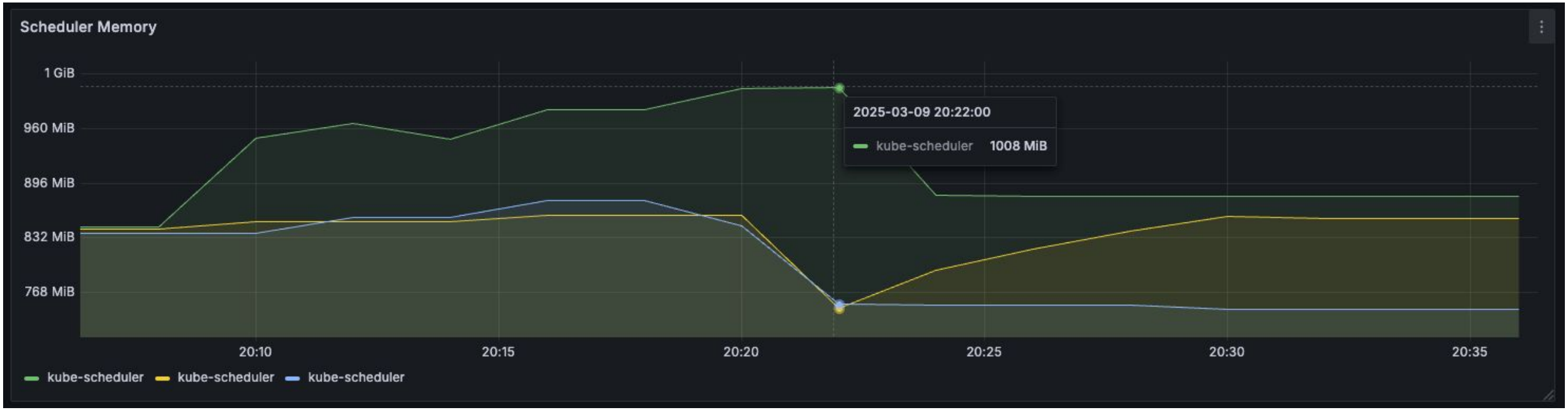
dra-scheduler Latency

Topo-aware memory usage



- Topo aware scheduler: ~570MB
- Kube Scheduler without DRA: ~770MB
- Total: ~1340MB

Kube Scheduler with DRA memory usage

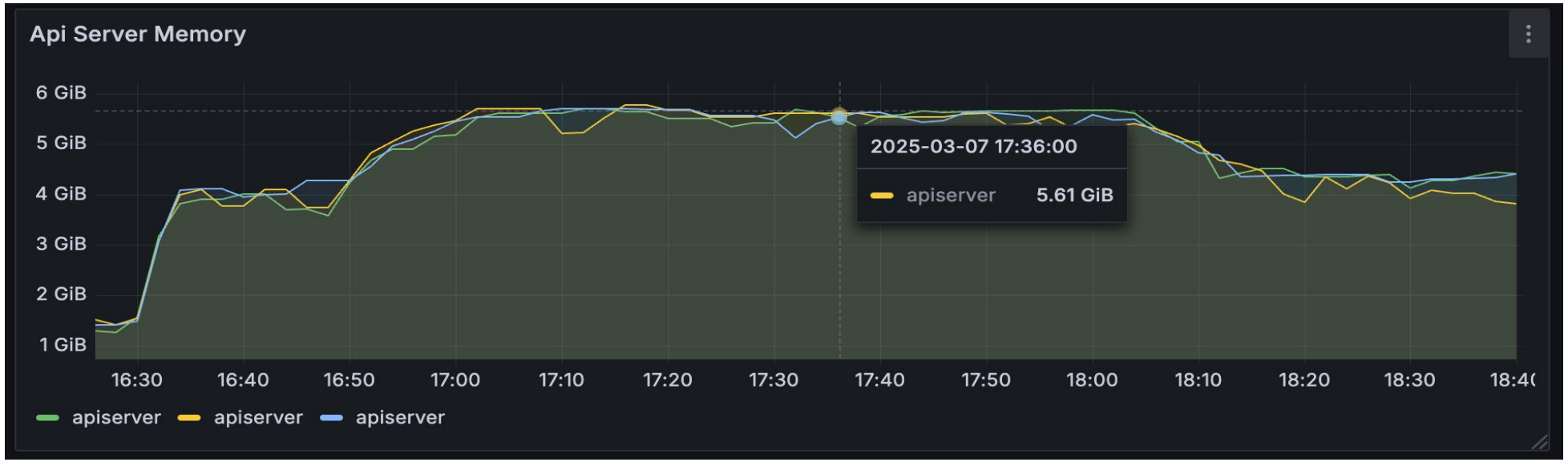


Total: ~1008MB

- Key Takeaways
  - DRA is improvement over legacy
  - Large portion of scheduler memory is just watch for data

Results: API Server memory

topo-aware-scheduler impact on apiserver



kube-scheduler with DRA impact on apiserver

Takeaways

- Our platform transformed from legacy to Cloud Native
- In this journey we made some mistakes, but we are able to handle the scale using some fining tuning techniques
- Kubernetes community has some nice upcoming efficiency gains
- DRA adoption is meeting our requirements for scale and performance

Leave us a review!

KubeCon CloudNativeCon Europe 2025

Q & A